Building Resilient Distributed Systems

PDC Assignment 4 | Baqir Hussain | BSCS23026

Part 1 — Root-Cause Analysis

Problem 1 — Lost Update (Synchronization)

The root cause is the complete absence of concurrency control in the document-update API. When User A and User B both send a `PUT /document/{id}` request at roughly the same time, FastAPI forwards both to the database as plain `UPDATE` statements. Because there is no version check, row-level lock, or optimistic-concurrency guard, whichever write lands second simply overwrites the first — silently, with a 200 OK to both callers. This is the classic *Lost Update* anomaly, a well-known race condition that arises whenever a read-modify-write cycle is not atomic. The bug lives at the intersection of the API handler (no version parameter) and the ORM query (no conditional WHERE clause).

Problem 2 — Dropped Webhook (Coordination)

Clerk delivers subscription-cancellation events via HTTP POST webhooks to our endpoint. The naive implementation acknowledges the event only when it succeeds; if a transient network failure or server restart drops the delivery, Clerk may retry a limited number of times but will eventually stop. Our backend never receives the cancellation, so the database row retains `is_premium = True` indefinitely. The fundamental issue is a lack of *guaranteed delivery*: the webhook pipeline is fire-and-forget with no persistent queue, no idempotency tracking, and no dead-letter mechanism to surface missed events. A single dropped HTTP request causes permanent state divergence between Clerk and our own user table.

Problem 3 — Blocking LLM Call (Fault Tolerance)

The LLM endpoint is called with a synchronous `requests.get()` / `httpx` call inside a FastAPI route handler. Although FastAPI is async-capable, a synchronous blocking call inside an async handler monopolises the event-loop thread. With a 60-second LLM timeout, even a single slow request can exhaust the thread pool and render the entire server unresponsive. There is also no circuit breaker: if the LLM service degrades, every incoming request dutifully waits the full 60 seconds before failing, causing cascading timeouts and making the LLM a single point of failure for the whole application.

Part 2 — Architectural Design

2.1 Optimistic Locking (Sync Fix)

Add an integer `version` column to the documents table (default 0). Every GET response includes the current version. Every PUT request must supply the client's known version. The server executes:

```
UPDATE documents SET content=:c, version=version+1 WHERE id=:id AND version=:v
```

If the UPDATE touches 0 rows the version was stale — another writer committed first. The server returns **HTTP 409 Conflict**, the client re-fetches the latest document and the user may merge or reapply their changes. No data is silently lost.

UML Sequence Diagram — Optimistic Locking

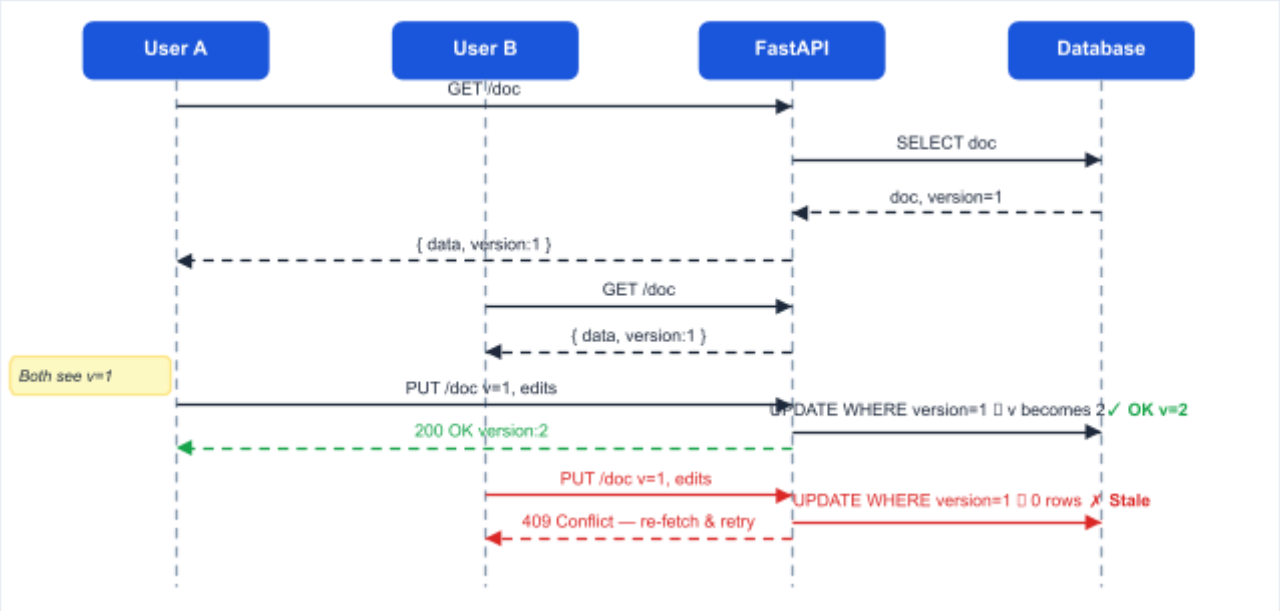

Figure 1 — User B's stale write is rejected (409); User A's write succeeds. No data is overwritten silently.

2.2 Fault-Tolerant Webhook Handler (Coordination Fix)

Three layers make webhook delivery reliable:

Layer	Mechanism
Idempotency Key	Store Clerk's event_id in a processed_events tab check before processing.
Durabl e Queue	On receipt, write event to DB/Redis queue; worker processes asynchronously.
Dead-Letter Queue (DLQ)	After N retries, move failed events to a DLQ; alert on-call engineer.

m processing.

t trail.

2.3 Circuit Breaker + Fallback (Fault Tolerance Fix)

Wrap every outbound LLM call in a Circuit Breaker with three states:

State	Behaviour	Transition
CLOSED (normal)	Requests pass through normally; failures are counted.	→ OPEN after N consecutive failures

OPEN (failing)	Requests are immediately rejected; fallback response returned.	→ HALF-OPEN after timeout T
HALF-OPEN (probing)	One probe request is allowed through.	→ CLOSED if success; OPEN if failure

The fallback response for a StudySync LLM feature could be a cached previous answer, a polite error message ("AI assistant is temporarily unavailable"), or a lightweight local model response -- never a 60-second hang.

CAP Theorem Trade-offs

CAP theorem states that a distributed system can guarantee at most two of: **Consistency** (every read sees the latest write), **Availability** (every request gets a response), and **Partition Tolerance** (system operates despite network splits). Since network partitions are unavoidable in practice, real systems must choose between C and A when a partition occurs.

In summary: the sync and coordination fixes lean CP (we prefer a small, visible error over silent data corruption), while the circuit breaker leans AP (we prefer a graceful degradation over a full outage). These choices directly reflect the PACELC extension of CAP: even without a partition, optimistic locking trades latency for consistency on conflict paths.